

Code-Layout, Readability and Reusability

Date 03 October 2026
Team ID COMICCRAFT-TEAM
Project Name ComicCrafter AI – AI-Powered Comic Strip Generator
Maximum Marks 2 Marks

Project Structure

Path	Purpose	Quality Observation
app.py	Streamlit interface and end-to-end workflow.	Clear top-level orchestration.
BACKEND/generate_panels.py	Gemini prompt, generation and panel parsing.	Encapsulated AI-text responsibility.
BACKEND/generate_image.py	ClipDrop API calls and style mapping.	Separate image-generation responsibility.
BACKEND/process_comic.py	Text wrapping, borders and 3×2 composition.	Reusable image-processing functions.
BACKEND/main.py	CLI version of the generation pipeline.	Provides an alternate execution path.
README.md	Project overview, setup and usage.	Supports repeatable onboarding.
requirements.txt	Dependency declaration.	Simple environment setup.

Readability

- Functions have descriptive names such as `generate_panels()`, `generate_images()` and `create_comic_strip_with_text()`.
- Style descriptions are centralized in dictionaries instead of repeated throughout the UI.
- The backend separates text generation, image generation and image composition.
- Docstrings explain the purpose of most reusable functions.

Reusability

- The panel-generation function can be reused by the Streamlit or CLI entry point.
- The image-generation module accepts panel data and a selected style.
- The comic processor accepts arbitrary six image paths and six text values.
- Environment-based API keys allow the same code to run across development environments.